The Essential of Chapter 1

DBMS internals cut down to what the two mock exams ask: which subsystem, which stage, who chooses the plan, how a plan tree runs, and the SQL you must write

Advanced Data Management

2026-10-29

How to read this document

This is chapter 1 with everything removed that neither mock exam nor the chapter 1–3 quiz touches. Six sections, each ending in a *Mock drill* box that says which mock question lives there and the shape of the answer the rubric wants. Read it in order: every term is defined before it is used. Chapter 1 carries few points on its own (Mock v2 A1 and C1, Mock 1 2.1 and 3.7, about 20 points over the two exams), but "planner", "binding", "build side" and "blocking" are the words every other question is written in. Numbers to carry: `customers` 10,000 rows in 73 pages, `orders` 500,000 rows in 3,185 pages, 8 KB pages.

Left out on purpose: §1.1 (the school that kept its data in files) and the services table, §2.2 (the connection manager beyond one line), §2.5 (querying the catalog), the syntax-tree figure of §3.3, the Python skeleton of `SeqScan` and `HashJoin` in §3.5, the "why keep logical and physical plans separate" box, the row/column layout figure of §4.2, §5.4 (subqueries, DDL and DML), the tools list of §6, and the history notes.

1 The Declarative Gap and the Five Subsystems

Concept: DBMS, declarative query, plan

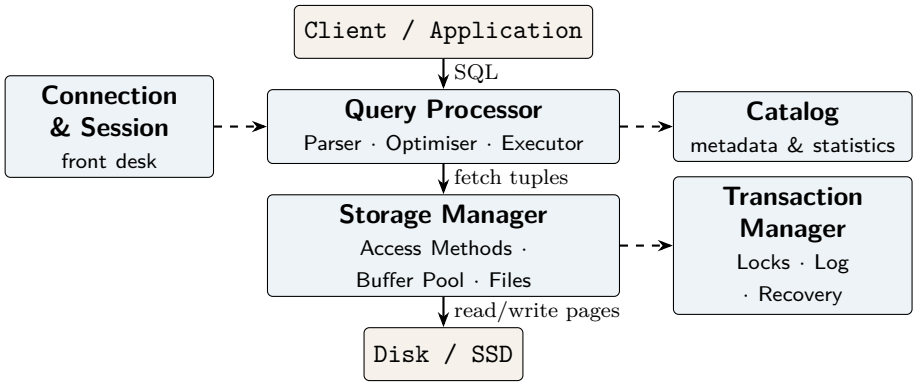
A **database management system** (DBMS) stores a collection of data and provides, as services, what applications over plain files had to build themselves: *querying* without a program per question, *concurrency control* so that two users updating at once lose nothing, *recovery* so that a crash leaves only committed work, *integrity* (keys, types, constraints), *security* and *efficient access* (indexes, caching, optimisation). The service that shapes this course is **declarative querying**: an SQL statement describes the *result* and says nothing about *how* to compute it. The DBMS must

therefore choose, for every query, a concrete procedure, the **plan**: scan or index, which join algorithm, in which order, from cache or from disk.

Key insight. SQL is a description of the answer. The plan is the recipe. Every mechanism in the course exists to turn the first into a good second.

1.1 Five subsystems

A DBMS is not one program but five cooperating parts (Hellerstein, Stonebraker and Hamilton, *Architecture of a Database System*, 2007). Knowing which part owns a problem tells you where to look, and the exam asks exactly that.



1. **Connection and session manager.** Authenticates clients, gives each session its own context, runs a pool of worker processes (PostgreSQL forks one per connection). No database work itself.
2. **Query processor.** The **parser** checks grammar, the **optimiser** (PostgreSQL's *planner*) chooses the cheapest plan, the **executor** runs it. §2–§4 live here.
3. **Storage manager.** Owns the bytes: rows in **pages** and files (chapter 2), **access methods**, that is indexes (chapter 3), and the **buffer pool**, the in-memory cache of disk pages (chapter 6). A disk is thousands of times slower than RAM, so this is where most performance is won or lost.
4. **Transaction manager.** Its **lock manager** coordinates concurrent transactions, its **log** records every change, and **recovery** replays or undoes changes after a crash: all of a transaction or none of it, even if the power fails halfway.
5. **Catalog.** The database's description of itself: tables, columns, types, indexes, and the **statistics** (row counts, value distributions) the optimiser lives on; stored as ordinary tables (`pg_class`, `pg_stats`).

Put a symptom in front of the list and name its owner: a Seq Scan where an Index Scan would be faster is the planner's; two concurrent updates with one change silently lost is the transaction manager's (lock manager); a low hit ratio is the storage manager's (buffer pool); "column `amuont` does not exist" is the query processor's binder reading the catalog; an estimate off by 1,000× is the catalog's stale statistics (`ANALYZE`).

1.2 Why the storage manager runs its own buffer pool

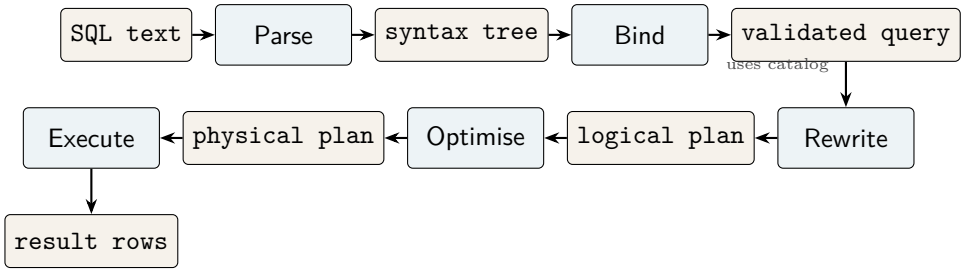
The operating system already caches file pages in RAM, yet every serious DBMS keeps its own **buffer pool**, because the storage manager knows three things the OS cannot. **The access pattern:** a scan touches each page once, an index root is touched by every lookup; under the OS's generic least-recently-used policy one large scan flushes every hot page (*sequential flooding*). **The write order durability needs:** the log record describing a change must reach disk *before* the changed data page; the OS writes dirty pages in any order, so after a crash the disk could hold a modified page with no record of how to undo it. **Pinning and memory accounting:** a page an operator is working on must not be paged out, memory is budgeted per operator (`work_mem`), and a page cached twice is *double buffering*. The full argument is *The Essential of Chapter 6*, §1.1.

Mock drill

Mock 1, 2.1 (7 pts) and **Quiz C6(a)**: the three reasons above, each with what breaks under the OS alone; the rubric asks for the durability one by name. **Quiz A1, C5(e)**: the five subsystems; a lost update belongs to the transaction manager (lock manager), not storage, not the query processor. Shape: numbered reasons, each "the DBMS knows X; the OS cannot, so Y breaks".

2 The Life of a Query, Stages 1–3: Parse, Bind, Rewrite

One SQL string goes in, rows come out. In between, five stages, each producing a more concrete object than the one before; the exam asks "*which* stage does X", so memorise the objects.



The running example, carried through every stage:

```
SELECT c.name, SUM(o.amount) AS total
FROM   customers c JOIN orders o ON o.cust_id = c.id
WHERE  c.country = 'MA'
GROUP BY c.name;
```

For every Moroccan customer, the value of their orders. A tiny instance: `customers(id, name, country)` holds (1, Sara, MA), (2, Amir, FR), (3,

Lina, MA); `orders(id, cust_id, amount)` holds (10, 1, 40), (11, 3, 15), (12, 1, 90), (13, 2, 70). Expected answer: Sara 130, Lina 15; Amir is French and is excluded.

2.1 Stage 1: parsing

Mechanism: Parser

Input: the SQL text. **Output:** a **syntax tree** mirroring the query's structure. A *lexer* splits the text into tokens (`SELECT`, `c.name`, `,`, ...); the parser checks the token sequence against the SQL grammar and builds the tree. Parsing is **pure syntax**: `FORM` instead of `FROM` is rejected here, but the parser has **no idea whether `customers` or `country` exist**: to it, `amuont` is just as good an identifier as `amount`. Whether there is such a table or column is a question for the next stage.

2.2 Stage 2: binding

Mechanism: Binder (semantic analysis)

Input: the syntax tree. **Output:** a **validated query** in which every name is resolved to a catalog object and every expression has a type: the query meets the **catalog**. Four checks: **existence** (do `customers`, `orders` and every column named exist?), **resolution** (`c` is `customers`; `c.country` is `text`), **type checking** (`SUM` over a numeric column; `c.name` may be selected because it is in `GROUP BY`), **permissions** (may this user read both tables?).

Key insight. "Column does not exist" and "relation does not exist" are *binding* errors, never parse errors. The parser cannot know; only the catalog can.

The output is the same tree with every leaf carrying a resolved reference and a type, like the type-decorated AST of a compiler. "relation `ordrs` does not exist" is raised here, after parsing succeeded.

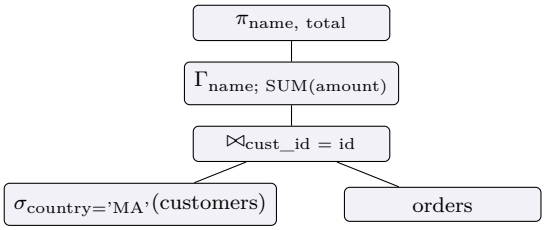
2.3 Stage 3: rewriting into a logical plan

Mechanism: Rewriter

Input: the validated query. **Output:** a tree of **relational-algebra** operators, the **logical plan**. It says *what* to compute, still not *how*: "join" names no algorithm, "customers" names no access path.

Relational algebra is the small set of operations every SQL query reduces to; four are enough here: **selection** σ_p keeps the rows satisfying predicate p (`WHERE`); **projection** $\pi_{a,b}$ keeps only columns a, b (the `SELECT` list); **join** $\bowtie_p$ pairs up rows of two inputs where p holds (`JOIN ... ON`); **grouping** $\Gamma_{g;\,f}$ groups by g and computes aggregate f per group (`GROUP BY`). The logical plan of the running example, read bottom-up: filter customers to Morocco, join with orders, group by name summing amount, project the two output

columns.



Concept: Predicate pushdown

Rule-based rewrites tidy the tree: views are expanded, subqueries flattened into joins. The one that matters most is visible above: the filter on `country` sits *below* the join, although the SQL wrote it in a `WHERE` that logically applies after the join. Moving a selection below a join is **predicate pushdown**: always *safe* when the predicate mentions only one input's columns (the join cannot change whether a customer is Moroccan), always *good* because fewer rows enter the join and every operator above does less work.

Mock drill

Quiz A2, A3, B5, C5(d): the five stages in order; `SELECT * FROM ordrs` fails at *binding* (answer b), because `ordrs` is a syntactically valid identifier and only the catalog lookup reveals there is no such table. **Quiz A9, C5(b)**: predicate pushdown, applied at the rewrite stage; the filter mentions only one table's columns, so it runs before the join and shrinks its input. Shape: name the stage, what it consults (the catalog), why the earlier stage could not do it.

3 Stage 4: the Optimiser and the Physical Plan

Mechanism: Optimiser (planner)

Input: the logical plan. **Output:** a **physical plan**: the same tree, but every operator replaced by a concrete *algorithm* and every table by a concrete *access path*. The optimiser estimates the cost of each alternative from the catalog's statistics and keeps the cheapest.

The logical plan left three decisions open, and the optimiser makes all three. **Access path:** read all of `customers` and test each row (**Seq Scan**), or use an index on `country` to jump to the Moroccan rows (**Index Scan**)? A cost comparison: a sequential page costs 1 unit, each row fetched through an index costs a random page read at 4 units, so the index pays only when few rows match (chapter 5). **Join algorithm:** nested loop, sort-merge or hash join (chapter 4), each best in a different situation (tiny outer with an index, sorted inputs, large equi-join). **Order:** filter the customers first and then join, or join and filter afterwards; with more tables, which pair first. One possible

outcome for the running example: **HashAggregate** (by name, SUM) over a **Hash Join** on `cust_id = id` whose children are a **Seq Scan** of `customers` with the filter `country = 'MA'` and a **Seq Scan** of `orders`: the logical tree with "join" replaced by "hash join" and "customers" by "sequential scan of customers" (the tree drawn in §4 has this shape). Many physical plans exist for one logical plan, and the optimiser picks a different one when the data grows or an index is added, without the SQL changing; with stale statistics the same SQL can suddenly run a hundred times slower.

In the real world. `EXPLAIN <query>` prints the plan PostgreSQL chose, with estimated rows and cost per node; `EXPLAIN (ANALYZE, BUFFERS)` runs it and adds actual rows, milliseconds and page counts.

Mock drill

Mock v2 A1 (3 pts): "which component decides Seq Scan versus Index Scan on `orders`?" The *planner* (answer b): the access path is a physical-plan choice made by costing both alternatives from the statistics (sequential pages at 1 against matching rows at random cost 4). Rule the others out: the parser only builds the syntax tree; the executor runs whatever plan it is given; the buffer manager serves page requests and never sees the SQL. **Quiz C5(c)**: an estimate of 4 rows against 9,000 actual is the planner working from stale statistics; fix with `ANALYZE events`.

4 Stage 5: Execution, the Iterator Model and Plan Trees

Mechanism: Executor and the Volcano / iterator model

Each operator of the physical plan is a small object with three methods: `open()` gets ready (allocate a hash table, open the scan, call `open()` on the children); `next()` returns the *next* output tuple, calling `next()` on the children as often as it needs, and returns nothing when exhausted; `close()` cleans up. The top operator is asked for tuples: **requests flow down the tree, tuples flow up**, one at a time.

Why this design? Uniformity: a hash join does not care whether its child is a scan or another join; it just calls `next()`. And **pipelining**: tuples flow one at a time, so the join starts producing while the scan is still reading, and intermediate results usually need not be written to a temporary table. On the tiny instance: `open()` on the join *drains* the customers scan into the hash table $\{1 \mapsto \text{Sara},\ 3 \mapsto \text{Lina}\}$ before a single order is read (Amir never leaves the scan); then each order is probed and passed up as it arrives, (Sara, 40), (Lina, 15), (Sara, 90), order 13 matching nothing; the aggregate emits (Sara, 130) and (Lina, 15) only once the join is exhausted.

Concept: Streaming and blocking operators

An operator is **streaming** (pipelined) if it can emit an output tuple after seeing only part of its input: a scan with a filter, a projection, the *probe* side of a hash join. It is **blocking** (a *pipeline breaker*) if it must consume its *whole* input before emitting anything: a sort, a hash aggregate, and

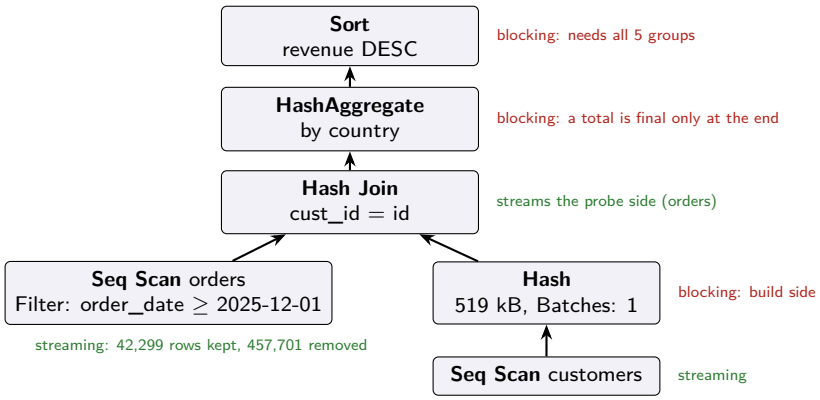
the **build** side of a hash join, where the `Hash` node reads every row of its child before the first probe. A blocking node at the top means the first row appears only at the very end; normal for any report with `GROUP BY` or `ORDER BY`.

4.1 Reading an EXPLAIN tree

Technique: Indentation is the tree

`EXPLAIN` prints the top node first and the children indented under it with `->`; the deepest-indented leaf runs first, so read bottom-up. Under a `Hash Join`, the child below the `Hash` node is the **build side** (read entirely into memory first); the other child is the **probe side** (streamed through). The December report of Mock v2, Part C:

```
Sort  (rows=5)   Sort Key: (sum(o.amount)) DESC
  ->  HashAggregate  (rows=5)   Group Key: c.country
        ->  Hash Join  (rows=43388)  (actual rows=42299)
              Hash Cond: (o.cust_id = c.id)
              ->  Seq Scan on orders o  (rows=43388) (actual rows=42299)
                    Filter: (order_date >= '2025-12-01')
                    Rows Removed by Filter: 457701
              ->  Hash  (Buckets: 16384  Batches: 1  Memory Usage: 519kB)
                    ->  Seq Scan on customers c  (rows=10000)
```



Mock drill

Mock v2 C1 (5 pts): draw the tree above and label every node with the reason in half a sentence: Sort (blocking), HashAggregate (blocking), Hash Join (streams the probe side once the table is built), Seq Scan orders (streaming, filter per row), Hash over Seq Scan customers (blocking: the build side is a pipeline breaker). Close with "two of five stream; the first row appears only at the end". **Quiz A5–A8, C5(a)**: `open/next/close`; pipelining and what breaks it; the deepest-indented line runs first; the child under `Hash` is the build side.

5 The SQL the Exam Makes You Write

Concept: Logical evaluation order

SQL is *written* `SELECT`-first but *evaluated* `FROM`-first: (1) `FROM` with its joins, where to read from; (2) `WHERE`, keep which rows; (3) `GROUP BY`, form groups; (4) `HAVING`, keep which groups; (5) `SELECT`, what to return, aliases are born here; (6) `ORDER BY`, final order. Consequences: an alias defined in `SELECT` **cannot be used in `WHERE`**, which runs before the alias exists, but can be used in `ORDER BY`; `WHERE` filters *rows* before grouping and may not contain an aggregate, `HAVING` filters *groups* after and is where `COUNT(*) > 100` goes; every selected column must be in `GROUP BY` or inside an aggregate.

Key insight. The order 1–6 is the rewriter's logical plan read bottom-up: FROM/JOIN is $\bowtie$, WHERE is σ, GROUP BY and HAVING are Γ and a σ above it, SELECT is π.

Worked example: Mock 1, 3.7(a): purchase events per country in the last 30 days

Tables `events(user_id, event_type, ts, ...)` and `users(user_id, country, ...)`; the exam date is 2026-10-12.

```
SELECT   u.country, count(*) AS purchases
FROM     events e JOIN users u ON u.user_id = e.user_id
WHERE    e.event_type = 'purchase'
  AND    e.ts >= date '2026-10-12' - interval '30 days'
GROUP BY u.country
ORDER BY purchases DESC;
```

Against the order: join (1), two row filters (2), one group per country (3), no group filter (4), country and count out (5), and the alias `purchases` is legal in `ORDER BY` because it runs last (6). Both filters mention only `events` columns, so the rewriter pushes them below the join.

Mock drill

Mock 1, 3.7(a) (2 pts of 4) is the query above; the points are for the join condition, the two filters in `WHERE` (not `HAVING`), `GROUP BY` on the non-aggregated column, and a sensible `ORDER BY`. Part (b), the `generate_series` insert, is Lab 5 material.

6 Workloads and Storage Models

Concept: OLTP and OLAP

OLTP (online transaction processing) runs the business: many short transactions, each touching a few rows and usually a whole record ("record this payment"). **OLAP** (online analytical processing) analyses it: few long queries scanning millions of rows but only a handful of columns ("revenue by region, last three years"). The two pull the storage layout

in opposite directions.

Take the table (1, Sara, 40), (2, Amir, 15), (3, Lina, 90). A **row store** writes it on disk record by record, `1,Sara,40 | 2,Amir,15 | 3,Lina,90`: all fields of one record sit together, so reading or updating a whole record touches one place, right for OLTP. A **column store** writes it column by column, `1,2,3 | Sara,Amir,Lina | 40,15,90`: a query on one column reads only that column's pages, and a run of values of one type compresses very well, right for OLAP. With 30 columns of similar width, `SELECT avg(price)` reads every page of the row store (each `price` sits beside 29 other fields) but about 1/30 of the pages of the column store, before compression. Rule of thumb: few rows, whole records ⇒ row store; many rows, few columns ⇒ column store.

Mock drill

Quiz A10: OLTP, many small reads and writes of whole rows, row storage; OLAP, few large scans of a few columns, column storage. **Quiz B6**: about 1/30 (answer b), one line on why. **Mock 1, 3.2(b)** "why is DuckDB on Parquet cheaper": the row scan reads every column of every row (8 GB) for one 8-byte `ts`; Parquet holds that column alone (50M × 8 bytes = 400 MB raw, tens of MB compressed).

Warning / Gotcha: The four mistakes that cost points

- "The parser will catch a misspelled column or table." It will not: parsing checks grammar only; the *binder*, consulting the catalog, catches it.
- "The executor (or the buffer manager) decides between Seq Scan and Index Scan." The planner decides, by cost, from statistics; the executor runs what it is given.
- Labelling the Hash Join blocking. The *build side* (the `Hash` node) blocks; the join itself streams the probe side.
- A `SELECT` alias or an aggregate in `WHERE`, which runs before `SELECT` and before grouping: aliases go in `ORDER BY`, aggregates in `HAVING`.

Summary: The essential in four lines

1. SQL says *what*; the DBMS chooses *how*. Five subsystems: connection manager, query processor (parser, planner, executor), storage manager (pages, indexes, buffer pool), transaction manager (locks, log, recovery), catalog.
2. Five stages: parse (grammar only) → bind (names, types, permissions, against the catalog) → rewrite (algebra, predicate pushdown) → optimise (access path, join algorithm, order, by cost from statistics: 1 per sequential page, 4 per random fetch) → execute. "Does not exist" is a binding error.
3. Iterator model: `open/next/close`, requests down, tuples up, pipelined. Streaming: scans, the probe side of a hash join. Blocking: Sort, HashAggregate, the `Hash` build side. Read `EXPLAIN` bottom-up.
4. SQL runs FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY: aliases only in ORDER BY, aggregates only in HAVING. OLTP: whole rows, row store. OLAP: few columns of many rows, column store (1/30 of the pages for one column of thirty).